

Name: Rushang Bagada

Roll no: u24cs076

1. Write a C++ program finding maximum and minimum numbers in an array using dynamic

memory allocation.

```
#include <iostream>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter the number of elements: ";
```

```
    cin >> n;
```

```
    int* arr = (int*)malloc(n * sizeof(int));
```

```
    if (arr == nullptr) {
```

```
        cout << "Memory allocation failed!" << endl;
```

```
        return 1;
```

```
    }
```

```
    cout << "Enter the elements: ";
```

```
    for (int i = 0; i < n; i++) {
```

```
        cin >> arr[i];
```

```
    }
```

```
    int max = arr[0];
```

```
    int min = arr[0];
```

```
    for (int i = 1; i < n; i++) {
```

```

        if (arr[i] > max) max = arr[i];

        if (arr[i] < min) min = arr[i];
    }

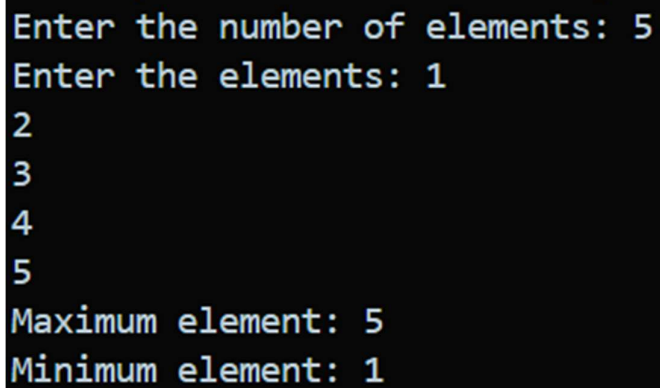
    cout << "Maximum element: " << max << endl;

    cout << "Minimum element: " << min << endl;

    free(arr);

    return 0;
}

```



The screenshot shows a terminal window with the following text:

```

Enter the number of elements: 5
Enter the elements: 1
2
3
4
5
Maximum element: 5
Minimum element: 1

```

2. Write a C++ program to display and calculate the sum of n numbers by using dynamic memory allocation function.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter the number of elements: ";
```

```
    cin >> n;
```

```

int* arr = new int[n];

int sum = 0;

cout << "Enter " << n << " numbers: ";

for (int i = 0; i < n; i++) {

    cin >> arr[i];

    sum += arr[i];

}

cout << "The sum is: " << sum << endl;

delete[] arr;

return 0;

}

```

```

Enter the number of elements: 5
Enter 5 numbers: 1 2 3 4 5
The sum is: 15

```

3. Write a C++ program to read a matrix of size $m \times n$ from the keyboard and display the same

on the screen using functions.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```

void readMatrix(int matrix[][10], int m, int n) {

    cout << "Enter elements of the matrix:" << endl;

    for (int i = 0; i < m; i++) {

```

```
        for (int j = 0; j < n; j++) {  
            cin >> matrix[i][j];  
        }  
    }  
}
```

```
void displayMatrix(int matrix[][10], int m, int n) {  
    cout << "Matrix elements are:" << endl;  
    for (int i = 0; i < m; i++) {  
        for (int j = 0; j < n; j++) {  
            cout << matrix[i][j] << " ";  
        }  
        cout << endl;  
    }  
}
```

```
int main() {  
    int matrix[10][10], m, n;  
  
    cout << "Enter number of rows and columns: ";  
    cin >> m >> n;  
  
    readMatrix(matrix, m, n);  
    displayMatrix(matrix, m, n);  
  
    return 0;  
}
```

```
Enter number of rows and columns: 2 2
Enter elements of the matrix:
1 2 3 4
Matrix elements are:
1 2
3 4
```

4. Write a C++ program to swap the values of two numbers by using

A. Call by value

B. Call by address

C. Call by reference variable

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void swapByValue(int a, int b) {
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
}
```

```
void swapByAddress(int* a, int* b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
void swapByReference(int& a, int& b) {
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
}
```

```
int main() {
```

```
    int x, y;
```

```
    cout << "Enter two numbers: ";
```

```
    cin >> x >> y;
```

```
    // Call by value
```

```
    swapByValue(x, y);
```

```
    cout << "After swap by value: x = " << x << ", y = " << y << endl;
```

```
    // Call by address
```

```
    swapByAddress(&x, &y);
```

```
    cout << "After swap by address: x = " << x << ", y = " << y << endl;
```

```
    // Call by reference
```

```
    swapByReference(x, y);
```

```
    cout << "After swap by reference: x = " << x << ", y = " << y << endl;
```

```
    return 0;
```

```
}
```

```
Enter two numbers: 5 6
After swap by value: x = 5, y = 6
After swap by address: x = 6, y = 5
After swap by reference: x = 5, y = 6
```

5. An electricity board charges the following rates to discourage large consumption of energy.

For the first 100 units -60P per unit

For next 200 units -80P per unit

Beyond 300 units -90P per unit

All users are charged a minimum of Rs. 50.00. If the total amount is more than Rs.

300.00 then an additional surcharge of 15% is added. Write a program to read the name of the user and number of units consumed and print out the charges with names.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
    string name;
```

```
    int units;
```

```
    float total;
```

```
    cout << "Enter your name: ";
```

```
    cin >> name;
```

```
    cout << "Enter number of units consumed: ";
```

```
    cin >> units;
```

```
    if (units <= 100) {
```

```
        total = units * 0.60;
```

```
    } else if (units <= 300) {
```

```
        total = 100 * 0.60 + (units - 100) * 0.80;
```

```
    } else {
```

```
        total = 100 * 0.60 + 200 * 0.80 + (units - 300) * 0.90;
```

```
    }
```

```
    if (total < 50) {
```

```

        total = 50;
    } else if (total > 300) {
        total += total * 0.15;
    }

    cout << "Electricity charges for " << name << " are: Rs. " << total << endl;

    return 0;
}

```

```

Enter your name: rushang
Enter number of units consumed: 500
Electricity charges for rushang are: Rs. 460

```

6. Write a C++ program to store and display the information of students using structure pointers.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
struct Student {
```

```
    string name;
```

```
    int rollNo;
```

```
    float marks;
```

```
};
```

```
int main() {
```

```
    Student* s = new Student;
```

```
    cout << "Enter student name: ";
```



```

cin >> s->name;

cout << "Enter roll number: ";

cin >> s->rollNo;

cout << "Enter marks: ";

cin >> s->marks;


cout << "\nStudent Information:\n";

cout << "Name: " << s->name << endl;

cout << "Roll No: " << s->rollNo << endl;

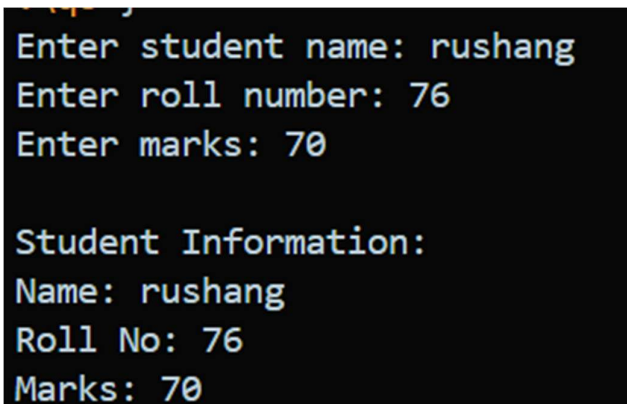
cout << "Marks: " << s->marks << endl;


delete s;

return 0;

}

```



```

Enter student name: rushang
Enter roll number: 76
Enter marks: 70

Student Information:
Name: rushang
Roll No: 76
Marks: 70

```

7. Write a C++ program to create three different integer pointers using malloc function and

new operator. Then use delete operator and free() function to delete the memory allocated to the above pointers.

For example:

```
int* ptr1 = (int*) malloc (sizeof(int));
```

```

int* ptr2 = new int;
int* ptr3 = NULL;
#include<bits/stdc++.h>
using namespace std;

int main() {

    int* ptr1 = (int*) malloc (sizeof(int));
    int* ptr2 = new int;
    int* ptr3 = NULL;

    *ptr1 = 10;
    *ptr2 = 20;

    cout << "Value of ptr1: " << *ptr1 << endl;
    cout << "Value of ptr2: " << *ptr2 << endl;

    free(ptr1);
    delete ptr2;

    return 0;
}

```

```

Value of ptr1: 10
Value of ptr2: 20

```

8. Write a C++ program solving a Tower of Hanoi Problem using a recursive

function. There are 3 disks in the tower 1

Shift top disk from tower 1 to tower 2; Shift top disk from tower 1 to tower 3;

Shift top disk from tower 2 to tower 3; Shift top disk from tower 1 to tower 2;

Shift top disk from tower 3 to tower 1; Shift top disk from tower 3 to tower 2;

Shift top disk from tower 1 to tower 2.

3 disks in tower 1 are shifted to tower 2.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void towerOfHanoi(int n, char source, char target, char auxiliary) {
```

```
    if (n == 1) {
```

```
        cout << "Move disk 1 from " << source << " to " << target << endl;
```

```
        return;
```

```
    }
```

```
    towerOfHanoi(n - 1, source, auxiliary, target);
```

```
    cout << "Move disk " << n << " from " << source << " to " << target << endl;
```

```
    towerOfHanoi(n - 1, auxiliary, target, source);
```

```
}
```

```
int main() {
```

```
    int n = 3;
```

```
    towerOfHanoi(n, 'A', 'C', 'B');
```

```
    return 0;
```

```
}
```

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

9. Create a class Student with data members name, roll_no, and a static data member count to

keep track of the number of student objects created.

A. Write a static member function to display the total number of students created.

B. Demonstrate the program by creating at least 3 objects.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
class Student {
```

```
    string name;
```

```
    int roll_no;
```

```
    static int count;
```

```
public:
```

```
    Student(string n, int r) : name(n), roll_no(r) {
```

```
        count++;
```

```
    }
```

```
    static void displayCount() {
```

```
        cout << "Total students created: " << count << endl;
```

```
    }
```

```
};
```

```
int Student::count = 0;
```

```
int main() {
```

```
    Student s1("Alice", 1);
```

```
    Student s2("Bob", 2);
```

```
    Student s3("Charlie", 3);
```

```
    Student::displayCount();
```

```
    return 0;
```

```
}
```

```
Total students created: 3
```

10. Create a class Book with the following:

A. Data members: title, author, and price.

B. A static data member totalBooks to keep track of the total number of books added.

A static member function showTotalBooks() to display the total number of books.

Write a program to demonstrate this by creating at least 4 Book objects, and show the details of each

book along with the total number of books created.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
class Book {
```

```
    string title;
```

```
    string author;
```

```
    float price;
```

```
static int totalBooks;
```

```
public:
```

```
Book(string t, string a, float p) : title(t), author(a), price(p) {  
    totalBooks++;  
}
```

```
static void showTotalBooks() {  
    cout << "Total books created: " << totalBooks << endl;  
}
```

```
void showDetails() {  
    cout << "Title: " << title << ", Author: " << author << ", Price: " << price << endl;  
}  
};
```

```
int Book::totalBooks = 0;
```

```
int main() {  
    Book b1("Book One", "Author A", 29.99);  
    Book b2("Book Two", "Author B", 39.99);  
    Book b3("Book Three", "Author C", 49.99);  
    Book b4("Book Four", "Author D", 59.99);  
  
    b1.showDetails();  
    b2.showDetails();  
    b3.showDetails();  
    b4.showDetails();  
}
```

```
Book::showTotalBooks();
```

```
return 0;
```

```
}
```

```
Title: Book One, Author: Author A, Price: 29.99  
Title: Book Two, Author: Author B, Price: 39.99  
Title: Book Three, Author: Author C, Price: 49.99  
Title: Book Four, Author: Author D, Price: 59.99  
Total books created: 4
```